



Guía de Colaboradores

Modelación Estadística

Maestría en Optimización y Modelación de Procesos
CIMAT Aguascalientes

Repositorio:

github.com/mop-cimat-ags/NotasModelacionEstadistica

Libro en línea:

mop-cimat-ags.github.io/NotasModelacionEstadistica

Catedráticos:

Dr. Sergio M. Nava Muñoz
Dr. Magín Zúñiga Estrada
Dr. Humberto Martínez Bautista

Compilación y edición: Vladimir Jiménez Pérez

14 de agosto de 2026

Índice de Contenido

Catedráticos colaboradores	4
Contenido	4
1. Herramientas necesarias	4
Herramientas comunes (todos)	5
Opción A — VS Code	5
Opción B — RStudio	5
Paquetes de R necesarios (todos)	5
2. Configurar Git y GitHub desde cero	6
2.1 Crear cuenta en GitHub	6
2.2 Verificar que Git está instalado	6
2.3 Configurar tu identidad en Git	6
2.4 Verificar que el editor detecta Git	7
2.5 Autenticación con GitHub — Token (PAT)	7
2.6 Alternativa: autenticación por SSH (avanzado)	8
3. Clonar y abrir el proyecto	8
Opción A — VS Code	8
Opción B — RStudio	9
Generar los datasets de la librería (solo la primera vez)	10
4. Estructura del proyecto	10
5. Renderizar localmente con Quarto	11
5.1 Vista previa en tiempo real (quarto preview)	11
5.2 Renderizado completo (quarto render)	11
5.3 Renderizar solo un formato	12
5.4 Renderizar un solo capítulo	12
5.5 El sistema de caché y freeze	12
5.6 Ver el resultado antes de subir	13
6. Cómo editar un capítulo	13
6.1 Antes de empezar — trae los cambios más recientes	13
6.2 Abre el archivo .qmd correspondiente	13
6.3 Escribe o edita el contenido	13
6.4 Verifica el resultado localmente	13
6.5 Sube los cambios	14
7. Cómo crear un capítulo nuevo	14
7.1 Crea el archivo .qmd	14

7.2 Regístralo en <code>_quarto.yml</code>	14
7.3 Convención de nombres de archivos	15
8. Sintaxis básica de Quarto	15
Encabezados	15
Texto con formato	15
Listas	15
Código R	16
Figuras	16
Ecuaciones	16
Tablas	17
Teoremas, definiciones y ejemplos	17
Referencias cruzadas	17
Notas al pie	17
Citas bibliográficas	18
Callouts (cajas destacadas)	18
9. Usar la librería <code>modelEstCIMAT</code>	18
Funciones disponibles	19
Datasets disponibles	20
10. Agregar funciones y datos a la librería	20
10.1 Agregar una función nueva	20
10.2 Agregar datos desde CSV	21
10.3 Agregar datos desde Excel	21
10.4 Documentar el dataset	21
11. Flujo de trabajo con Git	22
¿Qué es Pull?	22
Comandos esenciales	22
Mensajes de commit	23
Ver el historial de cambios	23
Deshacer cambios	23
12. Git en VS Code (sin terminal)	24
Abrir el panel de control de código fuente	24
Ver cambios	24
Hacer commit	24
Push y Pull	24
Trabajar con ramas en VS Code	24

Resolver conflictos en VS Code	24
13. Git en RStudio (sin terminal)	25
Abrir el panel Git	25
Flujo diario en RStudio	25
Resumen visual del panel	26
Resolver conflictos en RStudio	26
14. Trabajo con ramas (branches)	26
¿Por qué usar ramas?	26
Flujo recomendado	26
Comandos de ramas	27
Convención de nombres de ramas	27
Flujo completo paso a paso	27
Guardar trabajo sin hacer commit (stash)	28
15. Subir cambios a GitHub	28
Flujo estándar	28
Si hay conflicto	28
16. Preguntas frecuentes y errores comunes	29

Guía de referencia para catedráticos que contribuyen al libro **Modelación Estadística** de la Maestría en Optimización y Modelación de Procesos (OMP), CIMAT Aguascalientes.

Repositorio: <https://github.com/mop-cimat-ags/NotasModelacionEstadistica>

Libro en línea: <https://mop-cimat-ags.github.io/NotasModelacionEstadistica>

Catedráticos colaboradores

Nombre	Correo
Dr. Sergio M. Nava Muñoz	nava@cimat.mx
Dr. Magin Zúñiga Estrada	magin.zuniga@cimat.mx
Dr. Humberto Martínez Bautista	humberto.martinez@cimat.mx

Compilación y edición del repositorio: Vladimir Jiménez Pérez — vladimir.jimenez@cimat.mx

Contenido

1. Herramientas necesarias — comunes, VS Code, RStudio
2. Configurar Git y GitHub desde cero — VS Code y RStudio
3. Clonar y abrir el proyecto — VS Code y RStudio
4. Estructura del proyecto
5. Renderizar localmente con Quarto
6. Cómo editar un capítulo
7. Cómo crear un capítulo nuevo
8. Sintaxis básica de Quarto
9. Usar la librería modelEstCIMAT
10. Agregar funciones y datos a la librería
11. Flujo de trabajo con Git
12. Git en VS Code (sin terminal)
13. Git en RStudio (sin terminal)
14. Trabajo con ramas (branches)
15. Subir cambios a GitHub
16. Preguntas frecuentes y errores comunes

1. Herramientas necesarias

Instala **Git**, **R** y **Quarto** sin importar qué editor elijas. Luego instala solo el editor que vayas a usar — no es necesario tener ambos.

Herramientas comunes (todos)

Herramienta	Descarga	Para qué sirve
Git	https://git-scm.com/download/win	Control de versiones
R (≥ 4.1)	https://cran.r-project.org	Ejecutar código del libro
Quarto	https://quarto.org/docs/get-started	Renderizar el libro

Nota: Al instalar Git en Windows selecciona la opción “*Git from the command line and also from 3rd-party software*” para que cualquier editor lo detecte automáticamente.

Opción A — VS Code

Herramienta	Descarga
VS Code	https://code.visualstudio.com

Extensiones recomendadas (Ctrl+Shift+X):

- **Quarto** — soporte para archivos .qmd con previsualización
- **R** — resaltado de sintaxis y autocompletado para R
- **GitLens** — visualización avanzada de cambios e historial Git
- **GitHub Pull Requests** — revisar y crear Pull Requests desde VS Code

Opción B — RStudio

Herramienta	Descarga
RStudio	https://posit.co/download/rstudio-desktop

RStudio incluye una interfaz gráfica de Git integrada (panel Git) que permite hacer Pull, Commit y Push sin escribir ningún comando.

Paquetes de R necesarios (todos)

```
install.packages(c(
  "devtools", "knitr", "rmarkdown", "here",
  "usethis", "gitcreds"
), repos = "https://cloud.r-project.org")
```

2. Configurar Git y GitHub desde cero

Estos pasos solo se hacen una vez por computadora.

2.1 Crear cuenta en GitHub

1. Ve a <https://github.com> → clic en **Sign up**
2. Usa tu correo institucional `@ciamat.mx`
3. Verifica tu correo al terminar el registro
4. Comparte tu **nombre de usuario** con el coordinador del repositorio para que te den acceso

2.2 Verificar que Git está instalado

```
git --version  
# Debe mostrar: git version 2.xx.x
```

Si no aparece, reinstala Git y reinicia el editor.

2.3 Configurar tu identidad en Git

Git necesita saber quién eres para registrar cada cambio.

En VS Code — abre la terminal con `Ctrl+ñ` y ejecuta:

```
git config --global user.name "Tu Nombre Completo"  
git config --global user.email "tu.correo@ciamat.mx"  
  
# Verificar que quedó bien:  
git config --global --list
```

En RStudio — ejecuta en la consola de R:

```
usethis::use_git_config(  
  user.name = "Tu Nombre Completo",  
  user.email = "tu.correo@ciamat.mx"  
)
```

2.4 Verificar que el editor detecta Git

VS Code:

No requiere configuración extra si Git se instaló antes que VS Code. Verifica en el ícono de rama en la barra lateral (Ctrl+Shift+G) — debe mostrar los archivos del repositorio.

Para que Git use VS Code al escribir mensajes de commit:

```
git config --global core.editor "code --wait"
```

RStudio:

1. Ve a **Tools → Global Options → Git/SVN**
2. El campo *Git executable* debe mostrar la ruta, por ejemplo: C:/Program Files/Git/bin/git.exe
3. Si está vacío, clic en **Browse** → navega hasta git.exe
4. Reinicia RStudio

2.5 Autenticación con GitHub — Token (PAT)

GitHub no acepta contraseñas normales. Necesitas un token de acceso personal.

Crear el token:

Desde RStudio (más fácil):

```
usethis::create_github_token()
```

Abre directamente la página de GitHub con los permisos correctos marcados. Copia el token generado (ghp_XXXX...).

Manualmente en GitHub: Foto → **Settings** → **Developer settings** → **Personal access tokens** → **Tokens (classic)** → **Generate new token (classic)** → marca el permiso **repo** → genera y copia el token.

Guardar el token en VS Code / terminal:

```
git config --global credential.helper store
```

La primera vez que hagas `git push`, escribe tu usuario y pega el token como

contraseña. Git lo guarda automáticamente para las siguientes veces.

Guardar el token en RStudio:

```
gitcreds::gitcreds_set()  
# Pega el token cuando te lo pida
```

2.6 Alternativa: autenticación por SSH (avanzado)

SSH evita escribir credenciales y funciona con cualquier editor.

```
ssh-keygen -t ed25519 -C "tu.correo@cimat.mx" # presiona Enter en todo  
cat ~/.ssh/id_ed25519.pub                  # copia la clave completa
```

En GitHub → **Settings** → **SSH and GPG keys** → **New SSH key** → pega la clave.
Verifica:

```
ssh -T git@github.com  
# Respuesta esperada: Hi usuario! You've successfully authenticated...
```

Usa URL SSH al clonar:

```
git clone git@github.com:mop-cimat-agis/NotasModelacionEstadistica.git
```

3. Clonar y abrir el proyecto

Opción A — VS Code

Clonar el repositorio (solo la primera vez):

Abre la terminal (Ctrl+ñ) y ejecuta:

```
git clone https://github.com/mop-cimat-agis/NotasModelacionEstadistica.git  
cd NotasModelacionEstadistica  
code .
```

O desde VS Code: **File → Open Folder** y selecciona la carpeta clonada.

Volver a abrir el proyecto después de cerrarlo:

File → **Open Recent** → selecciona NotasModelacionEstadistica, o arrastra la carpeta al icono de VS Code.

Siempre haz **Pull** al iniciar (Ctrl+Shift+G → botón ↓) antes de editar.

Opción B — RStudio

Clonar el repositorio (solo la primera vez):

1. **File → New Project → Version Control → Git**
2. En *Repository URL* pega: `https://github.com/mop-cimat-ags/NotasModelacionEstadistica.git`
3. Elige la carpeta donde guardarlo → **Create Project**

RStudio clona el repositorio y activa el panel Git automáticamente.

Si el repositorio ya existe en tu computadora (clonado desde terminal o VS Code):

Crea el archivo de proyecto desde RStudio:

File → New Project → Existing Directory → Browse → selecciona la carpeta NotasModelacionEstadistica → **Create Project**

RStudio genera NotasModelacionEstadistica.Rproj y detecta el repositorio.

Si quieres subir el .Rproj al repositorio para que otros colaboradores también lo usen:

```
git add -f NotasModelacionEstadistica.Rproj
git commit -m "agrego proyecto RStudio"
git push
```

Volver a abrir el proyecto después de cerrarlo:

- File → **Recent Projects** → NotasModelacionEstadistica
- O doble clic en NotasModelacionEstadistica.Rproj en el explorador de Windows

Siempre haz **Pull** (flecha azul ↓ en el panel Git) al iniciar la sesión antes de editar.

Generar los datasets de la librería (solo la primera vez)

```
setwd("libreria")
source("data-raw/generar_datos.R")
setwd("../")
```

4. Estructura del proyecto

NotasModelacionEstadistica/

```
|
|— _quarto.yml                # Configuración del libro (no modificar)
|— _quarto-pdf.yml           # Perfil para renderizar solo PDF
|— _quarto-docx.yml          # Perfil para renderizar solo Word
|— index.qmd                 # Página de inicio del libro
|— referencias.qmd           # Capítulo de bibliografía
|— referencias.bib           # Referencias BibTeX
|— estilos-cimat.css         # Estilos HTML (no modificar)
|— GUIA_COLABORADORES.md    # Esta guía
|— README.md                 # Descripción del proyecto en GitHub
|
|— imagenes/
|   └─ logo-cimat.png
|
|— latex/                    # Archivos para el PDF (no modificar)
|   └─ preamble.tex
|   └─ portada.tex
|   └─ portada-docx.md
|
|— capitulos/
|   └─ parte1/                # Parte I – Probabilidad y Distribuciones
|       └─ cap01-intro-estadistica.qmd
|       └─ cap02-distribuciones.qmd
|       └─ cap03-distribucion-conjunta.qmd
|   └─ parte2/                # Parte II – Inferencia Estadística
|       └─ cap04-inferencia.qmd
|
|— libreria/                 # Paquete R: modelEstCIMAT
|   └─ R/                    # Funciones
|   └─ data/                 # Datasets (.rda)
|   └─ data-raw/             # Scripts para generar datasets
|   └─ tests/                # Pruebas unitarias
```

```
└─ .github/workflows/
  └─ publish.yml          # CI/CD automático (no modificar)
```

Regla: edita únicamente los archivos `.qmd` dentro de `capitulos/` y los archivos de `libreria/`. El resto lo administra el coordinador.

5. Renderizar localmente con Quarto

Antes de subir cualquier cambio a GitHub, verifica que el libro se ve bien en tu computadora. Quarto tiene dos modos principales.

5.1 Vista previa en tiempo real (`quarto preview`)

Es el modo que usarás mientras escribes. Abre una ventana del navegador que se actualiza automáticamente cada vez que guardas un archivo.

```
quarto preview
```

- El libro se abre en `http://localhost:4848` (o puerto similar)
- Cada vez que guardas un `.qmd`, el capítulo se re-renderiza en segundos
- Presiona `ctrl+c` en la terminal para detenerlo

Vista previa de un solo capítulo (más rápido):

```
quarto preview capitulos/partel/cap02-distribuciones.qmd
```

5.2 Renderizado completo (`quarto render`)

Genera todos los formatos (HTML, PDF, Word) y los guarda en `_book/`. Úsalo para verificar el resultado final antes de hacer push.

```
quarto render
```

El resultado queda en:

```
_book/
├─ index.html      ← libro HTML completo
├─ *.html          ← un archivo por capítulo
└─ Modelacion-Estadistica.pdf
```

└─ Modelacion-Estadistica.docx

5.3 Renderizar solo un formato

Para no esperar la compilación de PDF o Word cuando solo editas HTML:

```
quarto render --to html    # solo HTML
quarto render --to pdf     # solo PDF  → _book/
quarto render --to docx    # solo Word → _book/
```

O usando los perfiles del proyecto:

```
quarto render --profile pdf    # → _book-pdf/
quarto render --profile docx   # → _book-docx/
```

5.4 Renderizar un solo capítulo

```
quarto render capitulos/partel/cap04-inferencia.qmd
```

Esto genera solo ese capítulo en `_book/`. Útil para revisar rápido sin esperar que compile todo el libro.

5.5 El sistema de caché y freeze

Quarto tiene dos mecanismos para no re-ejecutar código innecesariamente:

Cache (`*_cache/`): guarda el resultado de cada chunk de R. Si el código no cambia, no lo vuelve a ejecutar. Se activa automáticamente con `cache: true` en `_quarto.yml`.

Freeze (`_freeze/`): guarda el resultado de archivos `.qmd` enteros. Si el archivo no cambió, Quarto lo omite completamente. Se activa con `freeze: auto`.

Por eso la primera vez que renderizas puede tardar varios minutos, pero las siguientes veces es mucho más rápido.

Forzar re-ejecución (cuando necesitas actualizar resultados):

```
quarto render --no-cache      # ignora el cache
quarto render --cache-refresh  # actualiza el cache
```

O borra manualmente la carpeta `_freeze/` para re-ejecutar todo.

5.6 Ver el resultado antes de subir

Una vez renderizado, abre `_book/index.html` directamente en el navegador para ver el libro exactamente como quedará publicado.

En VS Code puedes hacer clic derecho sobre `_book/index.html` → **Open with Live Server** (si tienes la extensión Live Server instalada).

6. Cómo editar un capítulo

6.1 Antes de empezar — trae los cambios más recientes

```
git pull
```

Hazlo **siempre** antes de empezar a editar para evitar conflictos.

6.2 Abre el archivo `.qmd` correspondiente

En VS Code, navega en el panel lateral a `capitulos/` y abre el archivo. Por ejemplo: `capitulos/partel/cap02-distribuciones.qmd`

6.3 Escribe o edita el contenido

Usa la sintaxis de Quarto (ver sección 8). Mientras escribes, activa la vista previa para ver los cambios al instante:

```
quarto preview
```

6.4 Verifica el resultado localmente

```
quarto render capitulos/partel/cap02-distribuciones.qmd
```

Revisa que no haya errores en la terminal y que el capítulo se ve bien.

6.5 Sube los cambios

```
git add capitulos/partel/cap02-distribuciones.qmd
git commit -m "actualizo ejemplos de distribucion normal en cap02"
git push
```

7. Cómo crear un capítulo nuevo

7.1 Crea el archivo .qmd

Dentro de la carpeta de la parte correspondiente:

capitulos/parte2/cap05-regresion-lineal.qmd

Plantilla mínima para empezar:

```
---
title: "Regresión Lineal Simple"
---

# Introducción

Texto de la introducción...

# Modelo

$$
Y = \backslash\text{beta}_0 + \backslash\text{beta}_1 X + \backslash\text{varepsilon}
$$
```

7.2 Regístralo en _quarto.yml

Avisa al coordinador para que agregue el capítulo al índice:

```
- part: "Parte II - Inferencia Estadística"
  chapters:
    - capitulos/parte2/cap05-regresion-lineal.qmd # nuevo
```

7.3 Convención de nombres de archivos

Usa siempre este formato: capNN-nombre-descriptivo.qmd

cap05-regresion-lineal.qmd	✓ correcto
cap05_RegresionLineal.qmd	✗ evitar mayúsculas y guiones bajos
Cap 05 Regresion.qmd	✗ evitar espacios

8. Sintaxis básica de Quarto

Encabezados

```
# Sección principal

# Subsección

# Sub-subsección
```

No uses # (título de nivel 1) — Quarto ya lo toma del title: en el YAML.

Texto con formato

```
**negrita**   *cursiva*   `codigo en línea`

> Esto es una cita o nota destacada.
```

Listas

```
- elemento 1
- elemento 2
  - subelemento

1. primer paso
2. segundo paso
```


Código R

```
```\r\
x <- c(1, 2, 3, 4, 5)
mean(x)
```\r\
```

Opciones útiles de chunk:

```
```\r\
#| echo: false # oculta el código, muestra solo el resultado
#| eval: false # muestra el código pero NO lo ejecuta
#| include: false # ejecuta pero no muestra nada (setup silencioso)
#| warning: false # oculta advertencias
#| fig-cap: "Mi figura"
#| fig-width: 7
#| fig-height: 4
plot(x)
```\r\
```

Figuras

```
![Descripción de la imagen](imagenes/mi-imagen.png){width=70%}
```

Ecuaciones

Ecuación en línea: $\mu = \frac{1}{n} \sum_{i=1}^n x_i$

Ecuación centrada:

```
$$
f(x) = \frac{1}{\sigma \sqrt{2\pi}} \exp\{-\frac{(x-\mu)^2}{2\sigma^2}\}
$$
```

Tablas

Variable	Media	Desv. Est.
---	---	---
Altura	168 cm	8 cm
Peso	68 kg	12 kg

Teoremas, definiciones y ejemplos

```

::: {#thm-grandes-numeros}
# Ley de los Grandes Números
Si  $X_1, X_2, \dots$  son iid con media  $\mu$ , entonces
 $\bar{X}_n \xrightarrow{p} \mu$ .
:::

::: {#def-esperanza}
# Esperanza Matemática
La esperanza de  $X$  se define como  $E[X] = \int x \cdot f(x) \, dx$ .
:::

```

Referencias cruzadas

Como establece el @thm-grandes-numeros, la media converge...
Ver la @def-esperanza para la definición formal.

Notas al pie

El estimador es consistente.^[1]

^[1]: Un estimador es consistente si converge en probabilidad al verdadero parámetro.

Citas bibliográficas

Agrega la referencia en `referencias.bib` y cítala:

```
Como demuestra @casella2002statistical [p. 234]...
```

Formato BibTeX en `referencias.bib`:

```
@book{casella2002statistical,  
  title    = {Statistical Inference},  
  author   = {Casella, George and Berger, Roger L.},  
  year     = {2002},  
  publisher = {Duxbury},  
  edition  = {2nd}  
}
```

Callouts (cajas destacadas)

```
::: {.callout-note}  
Esto es una nota informativa.  
:::  
  
::: {.callout-warning}  
Esto es una advertencia importante.  
:::  
  
::: {.callout-tip}  
Esto es un consejo o truco útil.  
:::
```

9. Usar la librería `modelEstCIMAT`

Al inicio de cada capítulo que use funciones de la librería, agrega este chunk de configuración:

```
``{r}  
#| include: false  
devtools::load_all(here::here("libreria"), quiet = TRUE)  
``
```

Funciones disponibles

```
# Estadística descriptiva  
resumen_estadistico(x)  
tabla_frecuencias(x, k = 6)  
curtosis(x)  
asimetria(x)  
momento_central(x, r = 3)  
coef_variacion(x)  
  
# Inferencia  
ic_media(x, nivel = 0.95)  
ic_media(x, nivel = 0.95, sigma = 2) # varianza conocida  
ic_proporcion(x = 30, n = 100)  
ic_varianza(x)  
prueba_z(x, mu0 = 0, sigma = 1, alternativa = "bilateral")  
emv_normal(x)  
emv_exponencial(x)  
emv_poisson(x)  
  
# Simulación  
simular_distribucion("normal", n = 500, mean = 0, sd = 1, seed = 1)  
simular_bootstrap(x, FUN = mean, B = 1000, seed = 1)  
simular_ic_bootstrap(x, FUN = median, B = 2000, metodo = "bca")  
monte_carlo_integral(function(x) x^2, a = 0, b = 1, N = 100000)  
  
# Gráficas  
graficar_densidad("t", df = 10, cola = "bilateral", alpha = 0.05)  
graficar_densidad("normal", mean = 0, sd = 1, a = -1.96, b = 1.96)  
graficar_ic(ic_media(x))
```

Datasets disponibles

```
data(alturas_estudiantes) # n=80, altura/peso de estudiantes
data(tiempos_falla)       # n=60, tiempos exponenciales (fallas)
data(calificaciones_curso) # n=35, tres exámenes correlacionados
data(conteos_defectos)    # n=50, conteos Poisson (manufactura)
data(temperaturas_mensual) # n=365, temperaturas diarias Guanajuato
```

10. Agregar funciones y datos a la librería

10.1 Agregar una función nueva

Paso 1 — Abre o crea un archivo en `libreria/R/`:

- `estadisticas.R` — estadística descriptiva
- `inferencia.R` — IC, pruebas, EMV
- `simulacion.R` — bootstrap, Monte Carlo

O crea uno nuevo, ej. `libreria/R/regresion.R`.

Paso 2 — Escribe la función con documentación roxygen2:

```
#' Título breve de la función
#'
#' Descripción más detallada.
#'
#' @param x Vector numérico de entrada.
#' @param nivel Nivel de confianza entre 0 y 1 (default: 0.95).
#' @return Descripción de lo que devuelve.
#' @examples
#' mi_funcion(c(1, 2, 3, 4, 5))
#' @export
mi_funcion <- function(x, nivel = 0.95) {
  mean(x) * nivel
}
```

La línea `@export` es obligatoria para que la función quede disponible.

Paso 3 — Agrega el nombre al `libreria/NAMESPACE`:

```
export(mi_funcion)
```

Paso 4 — Recarga y prueba:

```
devtools::load_all("libreria")
mi_funcion(c(10, 20, 30))
```

10.2 Agregar datos desde CSV

1. Copia tu archivo a libreria/data-raw/mis_datos.csv
2. Agrega al final de libreria/data-raw/generar_datos.R:

```
mis_datos <- read.csv("data-raw/mis_datos.csv")
save(mis_datos, file = "data/mis_datos.rda")
```

3. Ejecuta el script:

```
setwd("libreria")
source("data-raw/generar_datos.R")
setwd("../")
```

10.3 Agregar datos desde Excel

```
install.packages("readxl", repos = "https://cloud.r-project.org")

# En generar_datos.R:
mis_datos <- readxl::read_excel("data-raw/mis_datos.xlsx")
save(mis_datos, file = "data/mis_datos.rda")
```

10.4 Documentar el dataset

Agrega en libreria/R/datos.R:

```
#' Nombre descriptivo del dataset
#'  
#' @format Data frame con N filas y M columnas:
```

```
#' \describe{
#'   \item{columna1}{Descripción.}
#'   \item{columna2}{Descripción.}
#' }
#' @source Origen de los datos.
#' @examples
#' data(mis_datos)
#' head(mis_datos)
"mis_datos"
```

11. Flujo de trabajo con Git

Git registra todos los cambios del proyecto como fotografías (commits). Si algo sale mal, siempre puedes regresar a una versión anterior.

¿Qué es Pull?

Pull = descargar al tu computadora los cambios que existen en GitHub.

El repositorio en GitHub es la versión compartida del proyecto. Si otro colaborador subió cambios mientras tú no estabas trabajando, tu copia local quedó desactualizada. Pull trae esas actualizaciones para que estés al día.

GitHub —(pull)—> tu computadora

tu computadora —(push)—> GitHub

Por eso el flujo siempre es: **Pull** → **editar** → **Commit** → **Push**. Si editas sin hacer Pull primero y alguien más también editó los mismos archivos, puede surgir un **conflicto** que tendrás que resolver manualmente. La regla de oro: **Pull al iniciar**, **Push al terminar**.

Comandos esenciales

Comando	Qué hace
git pull	Trae los cambios más recientes de GitHub
git status	Muestra qué archivos modificaste
git diff archivo	Muestra exactamente qué cambió en un archivo
git add archivo.qmd	Marca un archivo para incluir en el commit
git add .	Marca todos los archivos modificados
git commit -m "mensaje"	Guarda los cambios con descripción
git push	Sube los cambios a GitHub
git log --oneline	Historial de commits resumido

Comando	Qué hace
<code>git restore archivo</code>	Descarta cambios no guardados en un archivo

Mensajes de commit

Escribe mensajes claros en español que describan qué cambiaste:

```
# Buenos mensajes
git commit -m "agrego seccion de distribuciones discretas en cap02"
git commit -m "corrijo formula de varianza muestral en cap01"
git commit -m "agrego dataset de precios de vivienda en libreria"

# Malos mensajes (evitar)
git commit -m "cambios"
git commit -m "fix"
git commit -m "update"
```

Ver el historial de cambios

```
git log --oneline          # resumen compacto
git log --oneline -10      # últimos 10 commits
git log --oneline --graph  # con gráfico de ramas
```

Deshacer cambios

```
# Descartar cambios no guardados en un archivo
git restore capitulos/partel/cap02-distribuciones.qmd

# Descartar TODOS los cambios no guardados (cuidado)
git restore .

# Sacar un archivo del área de staging (después de git add)
git restore --staged archivo.qmd
```


12. Git en VS Code (sin terminal)

VS Code tiene una interfaz gráfica completa para Git. Si prefieres no usar la terminal, puedes hacer todo desde aquí.

Abrir el panel de control de código fuente

Clic en el ícono de rama en la barra lateral izquierda (o `Ctrl+Shift+G`).

Ver cambios

Los archivos modificados aparecen en la sección **Changes**. Haz clic en cualquier archivo para ver exactamente qué líneas cambiaron (verde = agregado, rojo = eliminado).

Hacer commit

1. En el panel Source Control, escribe el mensaje en el campo de texto
2. Haz clic en el ícono  (**Commit**)
3. O presiona `Ctrl+Enter`

Push y Pull

En la barra inferior de VS Code aparece un ícono con flechas:  — tienes 1 commit listo para subir → clic para hacer push —  — hay 1 commit nuevo en GitHub → clic para hacer pull

También puedes ir al menú ... en el panel Source Control y elegir **Push** o **Pull**.

Trabajar con ramas en VS Code

- La rama actual aparece en la esquina inferior izquierda
- Clic en ella para cambiar de rama o crear una nueva
- Para crear: escribe el nombre y selecciona **Create new branch**

Resolver conflictos en VS Code

Cuando hay un conflicto, VS Code lo muestra con botones de ayuda:

- **Accept Current Change** — queda tu versión
- **Accept Incoming Change** — queda la versión de GitHub
- **Accept Both Changes** — combina las dos versiones

- **Compare Changes** — muestra las diferencias lado a lado

13. Git en RStudio (sin terminal)

RStudio tiene un panel Git integrado que permite hacer Pull, Commit y Push sin escribir ningún comando. Es la opción más sencilla para quien no está familiarizado con la terminal.

Abrir el panel Git

Al abrir el proyecto (`.Rproj`), el panel **Git** aparece automáticamente en la esquina superior derecha de RStudio (junto a Environment e History). Si no aparece, ve a **View → Show Git**.

Flujo diario en RStudio

1. Pull — traer cambios al iniciar la sesión

Haz clic en la **flecha azul ↓ (Pull)** en la barra del panel Git. Esto descarga los cambios que otros colaboradores hayan subido. Hazlo siempre antes de empezar a editar.

2. Editar los archivos `.qmd`

Abre el capítulo desde el panel *Files* (esquina inferior derecha) y edita normalmente.

3. Stage — marcar los cambios para guardar

Los archivos modificados aparecen en el panel Git con una **M** (modified) en la columna de estado. Marca la casilla ☐ de cada archivo que quieras incluir en el commit (equivale a `git add`).

4. Commit — guardar los cambios con descripción

Haz clic en el botón **Commit** en la barra del panel Git. Se abre una ventana con:
- Vista de los cambios (verde = líneas agregadas, rojo = eliminadas) - Campo de texto para el mensaje de commit

Escribe un mensaje descriptivo y haz clic en **Commit**.

5. Push — subir los cambios a GitHub

Haz clic en la **flecha verde ↑ (Push)**. Sube tus commits a GitHub. La primera vez te pedirá usuario y token (ver sección 2.6).

Resumen visual del panel

[Pull ↓] [Push ↑] [Commit] [Diff] [History]

✓ M capitulos/parte1/cap02-distribuciones.qmd
 ? archivo-nuevo-sin-rastrear.qmd

- **M** (modified) — archivo modificado
- **A** (added) — archivo nuevo ya en staging
- **?** — archivo nuevo sin rastrear (haz clic en  para agregarlo)
- **D** — archivo eliminado

Resolver conflictos en RStudio

Si al hacer Pull hay un conflicto, RStudio lo marca con **U** (unmerged). Abre el archivo — verás las marcas de conflicto:

```
<<<<<<< HEAD
Tu versión
=====
Versión de GitHub
>>>>>>> origin/main
```

Edita el archivo dejando solo la versión correcta, elimina las tres marcas (<<<<<<<, =====, >>>>>>>), haz Stage + Commit.

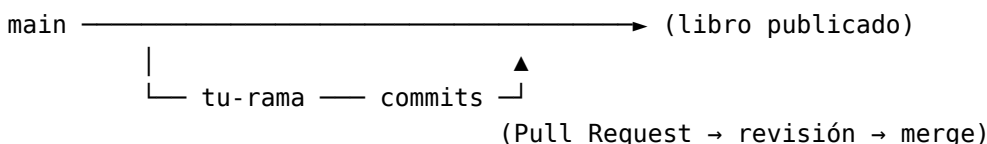
14. Trabajo con ramas (branches)

Una rama es una copia independiente del proyecto donde puedes trabajar sin afectar la versión principal (main).

¿Por qué usar ramas?

- Evitas sobrescribir el trabajo de otros colaboradores
- Tus cambios se revisan antes de integrarse al libro publicado
- Si algo sale mal, `main` sigue intacto

Flujo recomendado



Comandos de ramas

Comando	Qué hace
<code>git branch</code>	Lista ramas locales (* = rama actual)
<code>git branch -a</code>	Lista todas las ramas (locales y remotas)
<code>git checkout -b nombre-rama</code>	Crea y cambia de rama en un paso
<code>git checkout nombre-rama</code>	Cambia a una rama existente
<code>git push origin nombre-rama</code>	Sube la rama a GitHub
<code>git branch -d nombre-rama</code>	Elimina rama local (después del merge)

Convención de nombres de ramas

```
cap05-regresion-lineal      # nuevo capítulo
correccion-cap02-varianza   # corrección
dataset-precios-vivienda    # nuevo dataset
mejora-libreria-bootstrap   # mejora en la librería
```

Flujo completo paso a paso

```
# 1. Partir desde main actualizado
git checkout main
git pull

# 2. Crear tu rama
git checkout -b cap05-regresion-lineal

# 3. Trabajar – editar, renderizar localmente para verificar...

# 4. Guardar avances (puedes hacer varios commits)
git add .
git commit -m "agrego introduccion y modelo de regresion lineal"

git add .
git commit -m "agrego ejemplos con dataset alturas_estudiantes"

# 5. Subir la rama a GitHub
git push origin cap05-regresion-lineal

# 6. Abrir Pull Request en GitHub
#   → aparece un aviso amarillo en el repo
#   → clic en "Compare & pull request"
#   → describe los cambios y clic en "Create pull request"
#   → el coordinador revisa y hace merge
```

```
# 7. Después del merge, limpiar
git checkout main
git pull
git branch -d cap05-regresion-lineal
```

Guardar trabajo sin hacer commit (stash)

Si necesitas cambiar de rama pero no quieres hacer commit todavía:

```
git stash          # guarda temporalmente tus cambios
git checkout main  # cambia de rama
# ... haz lo que necesites ...
git checkout cap05-regresion-lineal
git stash pop      # recupera tus cambios
```

15. Subir cambios a GitHub

Flujo estándar

git pull	# 1. Trae últimos cambios
git status	# 2. Revisa qué modificaste
git add .	# 3. Marca los archivos
git commit -m "descripcion clara"	# 4. Guarda con mensaje
git push	# 5. Sube a GitHub

Al hacer push, GitHub Actions compilará y publicará el libro automáticamente en **2-3 minutos**.

Puedes ver el progreso en: <https://github.com/mop-cimat-ags/NotasModelacionEstadistica/actions>

Si hay conflicto

Un conflicto ocurre cuando dos personas modificaron el mismo archivo. Git te avisa con:

CONFLICT (content): Merge conflict in capitulos/partel/cap02.qmd

Abre el archivo en VS Code — verás las secciones en conflicto:

```
<<<<<<< HEAD
```

```
Tu versión del texto
```

```
=====
```

```
La versión que viene de GitHub
```

```
>>>>>>> origin/main
```

Edita el archivo dejando solo la versión correcta, elimina las marcas <<<<<<<, =====, >>>>>>> y luego:

```
git add capitulos/partel/cap02.qmd
git commit -m "resuelvo conflicto en cap02"
git push
```

Si el conflicto es complejo, consulta al coordinador antes de resolverlo.

16. Preguntas frecuentes y errores comunes

¿Cómo vuelvo a abrir el proyecto en RStudio si lo cerré?

File → **Recent Projects** → NotasModelacionEstadistica. O doble clic en el archivo NotasModelacionEstadistica.Rproj desde el explorador de Windows. Siempre haz Pull al inicio de la sesión antes de editar.

No tengo el archivo .Rproj porque el proyecto se creó en VS Code.

Créalo desde RStudio: **File** → **New Project** → **Existing Directory** → **Browse**, selecciona la carpeta NotasModelacionEstadistica y haz clic en **Create Project**. RStudio genera el .Rproj y detecta el repositorio Git automáticamente. Solo se hace una vez.

¿Por qué RStudio no muestra el panel Git?

Revisa que Git esté configurado: Tools → Global Options → Git/SVN → el campo *Git executable* debe tener la ruta de git.exe. Si está vacío, instala Git (<https://git-scm.com>) y reinicia RStudio.

¿Cómo veo el libro publicado?

En <https://mop-cimat-ags.github.io/NotasModelacionEstadistica>

¿Cuánto tarda en publicarse después de un push?

2 a 3 minutos. Progreso en: <https://github.com/mop-cimat-ags/NotasModelacionEstadistica/actions>

¿Puedo renderizar solo un capítulo localmente?

Sí:

```
quarto render capitulos/partel/cap02-distribuciones.qmd
```

¿Por qué quarto preview tarda tanto la primera vez?

Porque ejecuta todo el código R y construye el caché. Las siguientes veces será mucho más rápido gracias al sistema de freeze y caché.

¿Por qué me pide usuario y contraseña al hacer push?

GitHub no acepta contraseñas. Usa tu **Personal Access Token** (ver sección 2.5) como contraseña. Si ya lo configuraste antes y te vuelve a pedir, ejecuta `git config --global credential.helper store` y vuelve a intentar el push.

Error: remote: Repository not found

No tienes acceso al repositorio. Contacta al coordinador para que te agregue como colaborador en GitHub.

Error: error: failed to push some refs

Alguien más hizo push antes que tú. Primero haz `git pull`, resuelve conflictos si los hay, y luego `git push`.

Error: quarto: command not found

Quarto no está en el PATH. Reinstala Quarto y reinicia VS Code.

¿Puedo editar directamente en GitHub sin clonar?

Sí, para cambios pequeños: abre el archivo en GitHub → ícono del lápiz (Edit). No recomendado para cambios grandes ni para agregar imágenes.

¿Cómo agrego una imagen al capítulo?

1. Guarda la imagen en `imagenes/` o junto al capítulo 2. Refiérrela en el `.qmd`:

```
![Descripcion de la imagen](imagenes/mi-imagen.png){width=70%}
```

¿Qué hago si rompí algo y quiero volver atrás?

Si aún no hiciste commit:

```
git restore nombre-del-archivo.qmd # descarta cambios en ese archivo
git restore .                       # descarta TODOS los cambios
```

Si ya hiciste commit, consulta al coordinador — hay formas de revertir pero es mejor hacerlo con ayuda.

¿Qué es `_freeze/` y puedo borrarlo?

Es el caché de Quarto que guarda los resultados de los chunks. Si lo borras, Quarto volverá a ejecutar todo el código la próxima vez que renderices (tarda más). No es necesario borrarlo a menos que quieras forzar una actualización completa.